

title: “Fusão de sites 4G/5G, badges no mapa, múltiplos monitorings, clusters e visão geral de KPIs - Plan” type: feat date: 2026-08-19

Fusão de sites 4G/5G, badges no mapa, múltiplos monitorings, clusters e visão geral de KPIs

0. O que foi medido (base factual deste plano)

Nenhuma linha desta seção é inferida: tudo saiu do banco do evento que rodou em campo (`data/smart_events_testesantoamaro.db`), dos HARs de `har-5g-oss/` e do código atual.

0.1 A EP entrega o mesmo site físico duas vezes — uma por tecnologia

site_id	name	lat / lng	células
725483	SPSMG7	-23.640913 / -46.710655	12 × 4G-SPSMG7-*
1774059	SPSMG7	-23.640913 / -46.710655	3 × 5G-SPSMG7-*
725471	SPSMH1	-23.639723 / -46.721558	6 × 4G-SPSMH1-*
1774047	SPSMH1	-23.639723 / -46.721558	2 × 5G-SPSMH1-*
725469	SPSMH2	-23.639723 / -46.721558	10 × 4G-* (sem gêmeo 5G)

Mesmo nome, coordenada idêntica, id diferente. A causa está em `server.py:141`: `parse_sites` agrupa por `enodebid`, e o eNodeB 4G e o gNodeB 5G da mesma estação têm ids distintos. Consequências já visíveis na tela:

- a lista de sites mostra `SPSMG7` duas vezes, com valores diferentes;
- o mapa desenha dois marcadores na mesma coordenada;
- o popup mostra as células de apenas um dos gêmeos (`Células: 12` ou `Células: 2`, conforme qual ficou por cima — e isso mudou entre dois prints do mesmo dia).

0.2 As medições já estão corretamente segregadas — a fusão é de visualização

4G	CELL 171.950	SITE 19.324	(sites 725469 / 725471 / 725483)
5G_NRCELL	CELL 835	SITE 420	(sites 1774047 / 1774059)
5G_NRDUCCELL	CELL 3.841	SITE 1.530	(sites 1774047 / 1774059)

Os `site_id` de 4G e 5G são **disjuntos**. A coluna `technology` existe em toda linha, CELL e SITE. Portanto a fusão não migra dado, não altera coleta e não muda schema — é recorte de leitura. Isso é o que sustenta a exigência de “não interferir em nada nos processos de coleta”.

0.3 Bug pré-existente que a fusão expõe

[api/api.py:447-448](#):

```
for row in persisted_metric:
    metric_by_site[row["site_id"]] = row["value"]
```

`get_latest_site_kpi_by_metric` devolve uma linha por `(site_id, technology)` ([database.py:730](#)). O `dict` guarda a última, sem `ORDER BY`. Hoje isso não aparece porque cada `site_id` tem uma tecnologia só; no site fundido o valor exibido passaria a ser 4G ou 5G ao acaso. Precisa virar chave `(site, family)` junto com a fusão, não depois.

0.4 Por que a bola do VIP some

`_createMarker` ([map.js:205](#)) cria um `L.marker` por `site.id`, e `_markers` é chaveado por id — logo, dois marcadores na coordenada idêntica. O Leaflet ordena marcador pelo `y` da projeção; com a mesma latitude o z-index empata e quem fica por cima é a ordem de inserção no DOM. Como o badge do VIP é um `<circle>` dentro do `divIcon` ([map.js:369-379](#)), as pétalas do gêmeo pintam por cima dele.

Agrava: `resolve_site_id(cell)` liga o VIP ao site da célula servidora — VIP em célula 4G gruda em 725483, em célula 5G gruda em 1774059. O badge pode aparecer no gêmeo escondido.

0.5 O contrato do OSS é uma task por requisição

Nos 8 HARs capturados (dois OSS, 4G e 5G), sem exceção, o corpo é `[{"taskId": N, ...}]` — uma task por POST. O coletor hoje manda todas juntas ([collector.py:508](#)), premissa nunca medida. Isso coincide com a descrição operacional do processo manual: abrir várias tasks e coletar de cada uma.

Tamanho medido:

Task	Objetos	Requisição	Resposta
2242 (DU Cell, 12 contadores)	105	8.026 B	284.753 B
2225 (4G, 20 contadores)	116	4.848 B	279.199 B

≈ 2,4–2,7 KB de resposta por objeto por ciclo (o OSS acumula ~6 janelas). Com o limite de 300 células por monitoring, um evento como Curitiba (6.242 células) precisa de ~21 tasks:

desenho	requisição	resposta	timeout=30
21 tasks num POST (hoje)	~480 KB	15–17 MB	quebra
1 POST por task	~23 KB	~800 KB	folgado

0.6 Cada métrica tem a sua própria grade de tempo

Mesmo site, mesma janela, linhas `SITE` no banco de campo:

Sincronizar hover por índice entre 9 gráficos **exige eixo X comum**. Sem isso o índice 5 de um painel é 12:03 e o do outro é 12:47: o crosshair aponta horas diferentes e **a falha é silenciosa** — os gráficos parecem certos e o tooltip mente.

0.7 A conta dos 9 gráficos

Catálogo real: **4G = 13 definições**, das quais o bloco `Monitoring:` de `Fórmulas_4G_Monitoring(1).txt` são exatamente 9; **5G = 13** (`5G_NRCELL` 4 + `5G_NRDUCCELL` 9).

- **4G:** os 9 do bloco `Monitoring:` , 1:1, sem pareamento. `ran_rtt` / `terrestrial_rtt` (bloco `Adicionar:`) ficam **fora** — não têm contador na task 2225 e gerariam dois painéis vazios.
- **5G:** 13 métricas em 9 painéis, com **4 pares DL/UL** (decisão travada em 19/08).

0.8 O que já existe e será reaproveitado (nada de biblioteca nova)

- **Chart.js 4.4.0** + `chartjs-plugin-annotation` , vendorizados em [frontend/lib/](#).
- **Tooltip externo em HTML** já implementado como padrão do projeto: [vip.js:394-398](#), usado em [vip.js:591](#).
- **Modal de gráfico ampliado** (`#chart-popup-modal` + `_popupChart` , [kpi.js:398](#)) — molde da tela de visão geral.
- **Barra de modos + captura de clique no mapa** do cadastro (`polygonMode` , [index.html:1327](#), `updatePolygonOnMap` :[1540](#)) — molde do modo de clusters.
- `_applyVisibility` ([map.js:457](#)) já sabe esconder/mostrar marcador — base do filtro por cluster.
- `config_json` é blob ([database.py:236](#)) e `POST /api/events` grava o JSON recebido sem validar → clusters não exigem mudança de schema.
- **Não existe** helper de ponto-em-polígono no projeto (`autoCalculatePolygon` só faz bounding box) — serão ~10 linhas de ray casting.

0.9 Achado colateral: `throughput_d1` 5G voltou a calcular

O banco de campo tem **260 linhas CELL** de `throughput_d1` em `5G_NRDUCCELL` com valor real (`5G-SPSMH1-35-DA = 0,496` em `2026-08-19T12:06:00Z`). Como a fórmula exige os três contadores, `N.ThpVol.DL.LastSlot` **passou a existir na task 748**. O §0.3 do plano de 14/08 está resolvido.

0.10 Baseline da suíte

`pytest tests/ -q --basetemp=.pytest-work/tmp` → **332 passed, 10 skipped, 0 failed**. (O `MEMORY.md` ainda registra “261 passed, 3 failed” — desatualizado, corrigir junto.)

1. Decisões travadas por este plano

- **Chave de fusão: nome normalizado + coordenada.** Só funde quando o nome normalizado é igual e a distância entre as coordenadas é ≤ 50 m. Nos dados medidos as coordenadas são idênticas; a tolerância existe só para absorver diferença de arredondamento na EP. Nome igual em coordenada distante **não funde** e emite log — dois sites homônimos distintos não podem virar um.
- **O id do site fundido é sintético:** `name` normalizado (ex.: `SPSMG7`), com `members: [{site_id, family}]`. Nunca reaproveita o id de um dos gêmeos, que seria uma escolha arbitrária entre dois. Como `kpi_measurements.site_id` guarda os ids originais, **toda consulta de KPI expande fundido** → **membros**, explicitamente.
- **Uma única função de fusão** (`_merged_sites(config)`) em `api/api.py`, consumida por `get_sites`, `get_site_cells`, `get_alarms`, `get_vips` e `get_kpi_series`. Se um consumidor usar `config["sites"]` cru, ele devolve um `site_id` que nenhum marcador tem — falha silenciosa do mesmo formato das já registradas em `ERRORS.md`.
- **Cluster é dono da lista de sites (N:N), não propriedade do site.** Um site pode servir a mais de um cluster (o caso do estádio: um site atende Sul e Oeste).
- **O polígono do cluster é ferramenta de seleção, não regra de pertencimento.** Fica guardado para redesenhar, mas **nunca é reavaliado na exibição**; a verdade é `site_ids`.
- **O agregado de cluster se monta sobre as linhas `SITE` já persistidas dos membros**, nunca sobre `CELL` cru — mantém o número do cluster consistente com o número de cada site por construção, e é ordens de grandeza mais barato.
- **Uma task por POST** (§0.5). Unicidade passa a ser por `task_id`, não por tecnologia.
- **Nenhuma fórmula é criada, apagada ou reescrita** em nenhuma fase.
- **Grade de tempo comum** para os 9 gráficos, servida por um endpoint só (§0.6).

Ordem das fases e paralelismo



A Fase 1 vem primeiro porque **três das quatro fases restantes dependem dela** — adia-la adia tudo — e porque sozinha já corrige um defeito visível e entrega “uma linha por tecnologia”. A Fase 3 não toca em nenhum arquivo do frontend; **se o próximo evento passar de 300 células, ela sobe para primeira.**

Fase 1 — Fusão de sites 4G/5G e filtro por tecnologia

Explicação simples

Hoje a EP entrega o SPSMG7 duas vezes: uma linha com as células 4G e outra com as 5G, com ids diferentes. O resultado é o site aparecendo duplicado na lista e dois marcadores empilhados na mesma coordenada do mapa. Esta fase junta os dois num site só — pelo nome e pela coordenada — e acrescenta um seletor de tecnologia para o operador escolher se quer ver 4G, 5G ou os dois. Como as medições já vêm marcadas com a tecnologia, o gráfico passa a mostrar **uma linha por tecnologia** nas métricas que existem nas duas.

Escopo detalhado

api/api.py

- `_merged_sites(config) -> list[dict]` — nova, única fonte de fusão. Agrupa por (nome normalizado, coordenada dentro de 50 m) e devolve:

```
{ "id": "SPSMG7", "name": "SPSMG7", "lat": ..., "lng": ...,
  "members": [{ "site_id": "725483", "family": "4G"},
               { "site_id": "1774059", "family": "5G"}],
  "tech_families": ["4G", "5G"],
  "cells": [...],           # união das células dos membros
  "is_event_site": ... }
```

A família de cada membro sai das suas células via `_cell_technology_family` ([api.py:401](#)); membro sem família identificável entra como `None` e permanece visível em qualquer filtro (falha aberta). Nome igual com coordenada distante **não** funde e emite `logger.warning` com os dois ids.

- `get_sites` ([api.py:424](#)): itera sobre `_merged_sites`. Devolve `members` e `tech_families` no dicionário do site.
- **Corrigir \$0.3 na mesma fase:** `metric_by_site` passa a ser chaveado por `(merged_id, family)`; o valor exibido na lista respeita o filtro de tecnologia ativo e, em “Ambas”, segue a regra por métrica já usada em `get_kpi_series` ([api.py:645-655](#)) — soma para throughput/volume, média para availability/accessibility, máximo para utilização/usuários.
- `get_site_cells` ([api.py:505](#)): aceita o id fundido, devolve a união das células dos membros, cada uma com a sua `family`.
- `get_kpi_series` ([api.py:583](#)): aceita id fundido, **expande para os `site_id` dos membros** e ganha o parâmetro `technology_family` (`"4G"` | `"5G"` | `None`). Passa a devolver `series: [{technology, labels, values}]` — uma entrada por família presente — preservando `values / labels` no formato antigo quando há uma família só, para não quebrar chamadas existentes.
- `get_alarms` ([api.py:920](#)) e `get_vips`: `serving_site` passa a devolver o **id fundido**, via `_merged_sites`. Sem isso o badge da Fase 2 aponta para um id sem marcador.

frontend/js/state.js

- Campo `techFilter` (`"all"` | `"4G"` | `"5G"` , default `"all"`), no padrão dos demais.

frontend/index.html

- `<select id="tech-selector">` no `#chart-controls` (:229), antes do `#cell-selector`: `Ambas / 4G / 5G`. Fica oculto quando o site tem uma família só.

frontend/js/kpi.js

- `_populateCellSelector` (:326): filtra as células pela família ativa. É o que evita a explosão de linhas — SPSMG7 fundido tem 15 células, mas com “4G” selecionado o seletor lista 12 e com “5G” lista 3.
- `_refreshChart` (:568): consome `series[]` e cria um **dataset por tecnologia** quando o filtro está em “Ambas” e a métrica existe nas duas famílias. Cores fixas por família (4G e 5G), não a rotação de `CELL_COLORS`, para que a leitura seja estável entre métricas.
- `_renderSiteList` (:238): o site fundido aparece **uma vez**; quando o filtro é “Ambas”, o item mostra as famílias que ele tem.
- **Padrão mantido**: “Site completo” continua sendo o default e usa o agregado persistido — com a fusão ele passa a render 1 linha por tecnologia (2, no caso do SPSMG7), não 15.

frontend/js/map.js

- Nada estrutural: `renderSites` já cria um marcador por `site.id`, e com a fusão passam a ser **N marcadores para N sites físicos**. `_buildSectorIcon` desenha as pétalas 4G e 5G no mesmo ícone (as células já vêm unidas), respeitando `State.techFilter`.

frontend/js/bridge.js

- `MOCK_SITES` (:197) ganha um par de sites gêmeos 4G/5G com nome e coordenada iguais, para que o cenário mockado exercite a fusão sem VPN.

Testes

- `test_sites_com_mesmo_nome_e_coordenada_sao_fundidos` — o par `725483 / 1774059` vira um site SPSMG7 COM `tech_families == ["4G", "5G"]` e 15 células.
- `test_site_sem_gemeo_permanece_intacto` — `725469` (SPSMH2, só 4G) continua um site só.
- `test_nome_igual_em_coordenada_distante_nao_funde` — trava a regra dos 50 m; deve **falhar** se alguém reduzir a chave a só o nome.
- `test_valor_da_lista_nao_depends_da_ordem_das_linhas_site` — o teste central do §0.3: duas linhas SITE (4G e 5G) para a mesma métrica, em ordens invertidas, produzem o mesmo valor. Deve **falhar com a correção revertida**.
- `test_serie_do_site_fundido_traz_uma_entrada_por_tecnologia` — `get_kpi_series` no id fundido devolve `series` com 4G e 5G e **não mistura** as duas num `values` só.
- `test_alarme_e_vip_apontam_para_o_id_fundido` — `serving_site` de um alarme correlacionado a uma célula 5G devolve `SPSMG7`, não `1774059`. É o pré-requisito da Fase 2.
- `test_filtro_de_tecnologia_recorta_as_celulas` — `get_site_cells` com família `5G` devolve 3 e com `4G` devolve 12.
- Playwright, no padrão de `tests/test_frontend_collection_ui.py`: a lista de sites **não repete nome**; o `#tech-selector` alterna as células do `#cell-selector`.

Como validar

```
.venv\Scripts\python.exe -m pytest tests/test_api.py tests/test_frontend_collection_ui.py -v  
.venv\Scripts\python.exe -m pytest tests/ -q --basetemp=.pytest-work/tmp
```

Sem falha nova sobre as **332 passed / 10 skipped** do baseline (\$0.10).

Visualmente, com o evento `testesantoamaro`:

1. A lista de sites mostra **SPSMG7 e SPSMH1 uma vez cada** (hoje mostra duas).
2. Clicar em SPSMG7 no mapa abre um popup só, com **15 células** (hoje mostra 12 ou 2).
3. Com `#tech-selector` em "Ambas" e métrica `DL PRB Utility`, o gráfico traz **duas linhas**, rotuladas 4G e 5G.
4. Trocar para "4G": o seletor de células passa a listar 12; para "5G", 3.
5. O valor da coluna da lista **não muda** ao recarregar a página (prova do \$0.3).

Sugestão de commit

```
feat: merge 4G/5G twin sites and split KPI series by technology
```

Fase 2 — Badges de VIP e de alarme no marcador do site

Explicação simples

Com um marcador por site físico, a bolinha do VIP deixa de ser tapada pelo gêmeo. Esta fase garante que ela fique sempre por cima e acrescenta, do outro lado do marcador, um triângulo de exclamação quando o site tem alarme ativo — VIP à direita, alarme à esquerda.

Escopo detalhado

```
frontend/js/map.js
```

- **Pane dedicado:** `_map.createPane("badges")` com `zIndex` acima do `markerPane`, criado no `initMap` (:69). Garante a ordem mesmo se dois sites reais ficarem próximos no zoom afastado — a fusão resolve o empate exato, o pane resolve a sobreposição por vizinhança.
- **Triângulo de alarme:** simétrico ao badge de VIP, em `cx - Math.max(7, 11*scale)` (o VIP já está em `cx + ...`, :373). Cor pela maior severidade do site, reaproveitando a paleta de `STATUS_COLORS` /severidade de alarme já existente.
- **Corrigir o recorte:** `size` vem de `maxR + highlightDist` (:337). Com dois badges, `highlightDist` precisa considerar **a maior das duas extensões laterais**, senão o `viewBox` corta o ícone. Este é o defeito mais provável desta fase — o teste abaixo existe por causa dele.
- **Reatividade:** somar `State.on("change:alarms", () => renderSites(State.sites))` ao lado do `change:vips` já existente (:117).
- **Fonte do alarme:** `State.alarms` filtrado por `serving_site === site.id` — que só funciona porque a Fase 1 fez `get_alarms` devolver o id fundido.

frontend/js/bridge.js

- Cenário mockado com um site que tem VIP e alarme ao mesmo tempo, para exercitar os dois badges no mesmo ícone.

Testes

- `test_badge_de_vip_e_alarme_cabem_no_viewbox` — Playwright: mede o `<svg>` do marcador e afirma que os dois badges estão **dentro** dos limites. Deve **falhar** se o cálculo de `size` for revertido.
- `test_triangulo_aparece_so_com_alarme_no_site` — site sem alarme não tem o elemento.
- `test_badges_reagem_a_mudanca_de_alarms` — publicar `State.alarms` re-renderiza o marcador sem recarregar a página (regressão do listener esquecido).

Como validar

```
.venv\Scripts\python.exe -m pytest tests/test_frontend_collection_ui.py -v
```

Visualmente:

1. Num site com VIP, a bolinha dourada fica **visível e por cima** em qualquer zoom.
2. Num site com alarme, o triângulo aparece **à esquerda**, com a bolinha do VIP à direita.
3. Afastar o zoom até o mínimo: nenhum dos dois ícones é cortado.
4. Clicar no alarme no painel lateral foca o site e o triângulo é o do site focado.

Sugestão de commit

```
feat: keep VIP badge above markers and add alarm triangle to sites
```

Fase 3 — Múltiplos monitorings: uma requisição por task

Explicação simples

Cada monitoring do iManager cabe 300 células, então um evento grande precisa de várias tasks da mesma tecnologia. Hoje o sistema recusa mais de uma task por tecnologia e, quando aceita várias, manda todas numa requisição só — o que estouraria o tempo limite num evento grande. Esta fase libera N tasks por tecnologia e passa a consultar **uma task por requisição**, que é exatamente como o processo é feito manualmente e o único formato que o OSS foi observado respondendo.

Escopo detalhado

core/collector.py

- `_configured_pm_tasks` (:373): remover a trava “uma task por tecnologia”; passar a **rejeitar `task_id` repetido**. A ambiguidade real é o mesmo `task_id` declarado com duas tecnologias — o dict de :521 perderia uma delas em silêncio.
- `_collect_kpis_v2` (:490): trocar o POST único por **um POST por task, sequencial**, agregando `rows`, `cursors`, `received`, `unmapped`, `unmapped_cells`, `invalid`, `diagnostics` e `latest_data_at`. Regras:
 - **renovação de sessão uma vez por ciclo**, não por task — hoje o retry envolve a requisição única; envolver cada task multiplicaria renovações por 21;
 - **falha de uma task não derruba as outras**: vira diagnóstico daquela task e o ciclo segue, fechando `partial`;
 - `partial` / `data` / `empty` decididos **sobre o agregado**, não sobre a última task.
- `_log_monitoring_cycle` (:464): uma linha por task (`task=749 HTTP=200 recebidos=...` mapeados=... `invalidos=...`) mais uma linha de total do ciclo. Com 21 tasks, um total sem detalhe não permite achar a task que falhou.
- **Deduplicar diagnósticos por** `(metric, code)` antes de sair do parser (:1601). Deixa de ser opcional: 21 tasks × 300 objetos × ~6 janelas com um contador ausente gera dezenas de milhares de entradas por ciclo. O plano de 14/08 já apontava isso como item separado; nesta escala é requisito.
- **Guarda de tempo**: medir a duração do ciclo e emitir WARNING se ultrapassar `INTERVAL_KPI_SECONDS` (`scheduler.py:19`, 120 s). 21 tasks a ~1–3 s cabem, mas o ciclo precisa avisar quando não couber em vez de atrasar em silêncio.
- **Sem mudança** em `_obj_to_cell`, checkpoints, cursores, `_request_objects_for_task` ou `_expected_pm_cell_ids`: já são todos chaveados por `task_id`.

server_frontend/index.html

- Os três inputs fixos (`#pm-task-4g` , `#pm-task-nrcell` , `#pm-task-nrducell` , :766-775) viram **lista repetível** de linhas `{tecnologia, task_id}` com adicionar/remover.
- `editEvent` (:1769-1776): hoje mapeia `tech` → um `id` de `input` . **Esta mudança é obrigatória junto com a do backend** — no instante em que duas tasks da mesma tecnologia forem aceitas, reeditar um evento apagaria a segunda em silêncio.
- `renderEvents` (:1590): rótulo passa a contar as tasks por tecnologia (`4G × 3 · NR Cell × 1`).
- Evento legado com `pm_task_id` continua populando uma única linha 4G, sem reescrever nada até salvar.

core/kpi_formulas.py

- `throughput_d1` de `5G_NRDUCELL` (:152): `production_ready` para `True` e unidade medida, fechando o §0.9. Mudança de rótulo apenas — **a fórmula não muda**.

Testes

- `test_duas_tasks_da_mesma_tecnologia_sao_aceitas` — `[[{4G,747},{4G,752},{NRCELL,749}]` resolve para três tasks.
- `test_task_id_repetido_e_rejeitado` — mesmo `task_id` duas vezes levanta `ValueError` visível como `CollectionResult.error` , não como exceção que mata o scheduler.
- `test_cada_task_vira_uma_requisicao` — três tasks configuradas produzem **três POSTs**, cada um com um `taskId` só. Deve **falhar com o POST único revertido**.
- `test_falha_de_uma_task_nao_derruba_as_outras` — a segunda task responde 500; as linhas da primeira e da terceira são persistidas e o ciclo fecha `partial` com diagnóstico da segunda.
- `test_sessao_expirada_renova_uma_vez_por_ciclo` — trava a regra de renovação única.
- `test_diagnostics_sao_deduplicados_por_metrica_e_codigo` — um contador ausente em 100 objetos gera **uma** entrada, não 100.
- `test_cursors_de_tasks_distintas_nao_se_misturam` — regressão explícita sobre a chave `(task_id, object_key)` com duas tasks da mesma tecnologia.

Como validar

```
.venv\Scripts\python.exe -m pytest tests/test_kpi_monitoring.py tests/test_collector.py -v
.venv\Scripts\python.exe -m pytest tests/ -q --basetemp=.pytest-work/tmp
```

Em campo: cadastrar um evento com **duas tasks 4G** e rodar. Critério de saída, lido do log:

- uma linha `[monitoring] task=<id>` por task, todas com `HTTP=200` ;
- o total do ciclo soma os recebidos das duas;
- nenhum WARNING de duração de ciclo;
- `nao_mapeados` compatível com o inventário do evento.

Sugestão de commit

feat: query each PM task in its own request and allow several per technology

Fase 4 — Clusters de sites

Explicação simples

Clusters são grupos de sites criados pelo operador para analisar uma parte do evento — a arquibancada sul de um estádio, por exemplo. Um mesmo site pode estar em vários clusters, porque na prática ele atende mais de um setor. A montagem é feita no cadastro, de três formas que se somam: laço no mapa, clique no marcador e busca por nome. No app, um dropdown filtra a lista e o mapa pelo cluster escolhido.

Escopo detalhado

Modelo (sem mudança de schema, §0.8)

```
event.clusters = [
  { "id": "sul", "name": "Arquibancada Sul", "color": "#F85149",
    "site_ids": ["SPSMG7", "SPSMH1"],
    "polygon": [[lat,lng], ...] } // opcional, só para redesenhar
]
```

`site_ids` guarda **ids fundidos** (Fase 1) — o operador seleciona “SPSMG7” e leva 4G e 5G junto, sem a chance de marcar um e esquecer o outro.

server.py

- `parse_sites` (:117): somar `'cluster': ['cluster', 'grupo', 'agrupamento', 'setor', 'area', 'área']` ao `col_mappings`, **fora** da lista `required` — coluna opcional de verdade. Valores separados por `;` criam vários clusters para a mesma linha, preservando o N:N. A importação apenas **semeia**; depois disso a interface é a fonte de verdade.

server_frontend/index.html

- Terceiro modo na barra de ferramentas do mapa, “**Clusters**”, no padrão de `toolAutoPoly / toolDrawPoly` (:1488-1502).
- Painel de clusters: lista com nome, cor, contagem de sites, `+ Novo cluster`, renomear, excluir.
- Com um cluster ativo, três formas **que se somam**:
 1. **laço no mapa** — polígono desenhado; os sites dentro entram (ray casting, ~10 linhas, §0.8). Dois laços que se cruzam produzem sites compartilhados sem tratamento especial;
 2. **clique no marcador** — alterna dentro/fora. É a válvula do “não é estrito”;
 3. **busca por nome** — autocomplete para adicionar em lote (necessário em eventos com milhares de sites, onde o laço não resolve sozinho).
- Feedback visual: no modo Clusters, o marcador dos sites do cluster ativo ganha anel na cor dele e o painel mostra `Sul · 14 sites`. Fora do modo, marcadores voltam ao normal.
- Submit (:1662): `clusters` entra no payload ao lado de `integration`. `editEvent` repopula a partir de `event.clusters`.

`api/api.py`

- `get_sites`: cada site fundido ganha `cluster_ids: [...]`, derivado de `event.clusters`.
- `get_clusters(event_id)` — lista para o dropdown do app.
- `get_kpi_series`: aceita `scope="cluster"` + `scope_id`. Expande cluster → sites fundidos → `site_id` dos membros, lê as linhas `SITE` de cada um e combina entre sites com a **mesma regra por métrica** já usada ([api.py:645-655](#)). Não lê `CELL` cru.

`frontend/`

- `state.js`: `clusterFilter` (`"all" | <cluster_id>`).
- `index.html`: `<select id="cluster-selector">` no cabeçalho do `#site-panel` (:210), como no esboço.
- `kpi.js`: `_renderSiteList` respeita `clusterFilter`; o cluster aparece como opção de escopo do gráfico, ao lado dos sites.
- `map.js`: `_applyVisibility` (:457) passa a considerar o cluster; com um cluster ativo, o polígono dele é desenhado na cor do cluster.

Testes

- `test_cluster_aceita_o_mesmo_site_em_mais_de_um_grupo` — trava o N:N; deve **falhar** se alguém reintroduzir `site.cluster`.
- `test_poligono_nao_altera_pertencimento_depois_de_salvo` — mover um site para fora do polígono salvo **não** o remove do cluster (trava a decisão do §1).
- `test_coluna_cluster_da_ep_semeia_varios_grupos` — célula `Sul;Oeste` cria dois clusters com o site nos dois.
- `test_coluna_cluster_ausente_nao_quebra_importacao` — EP sem a coluna importa normalmente.
- `test_serie_de_cluster_combina_linhas_site_dos_membros` — o agregado do cluster de dois sites bate com a combinação manual das séries `SITE` deles, pela regra da métrica.
- `test_cluster_com_site_fundido_cobre_as_duas_tecnologias` — cluster com `SPSMG7` devolve série 4G e 5G.
- Playwright: criar cluster por laço, adicionar site por clique, filtrar no app e conferir a lista.

Como validar

```
.venv\Scripts\python.exe -m pytest tests/test_api.py tests/test_server_frontend_kpi_config.py -v  
.venv\Scripts\python.exe -m pytest tests/ -q --basetemp=.pytest-work/tmp
```

Visualmente, no cadastro:

1. Entrar no modo Clusters, criar “Sul”, desenhar um laço → os sites dentro ganham o anel e o contador sobe.
2. Criar “Oeste” com um laço que **crusa** o primeiro → o site da interseção fica nos dois (conferir contagem nos dois painéis).
3. Clicar num marcador fora do laço → entra; clicar de novo → sai.
4. Salvar, reabrir o evento → clusters e polígonos voltam preenchidos.

No app: 5. O dropdown CLUSTER lista “Todos / Sul / Oeste”; escolher “Sul” recorta a lista e o mapa. 6. Selecionar o cluster como escopo do gráfico → série agregada, com linha 4G e 5G.

Sugestão de commit

```
feat: create site clusters by map lasso and filter the dashboard by them
```

Fase 5 — Tela de visão geral com 9 gráficos sincronizados

Explicação simples

Uma tela nova, aberta pelo botão “Ver KPIs”, com nove gráficos ao mesmo tempo — os nove KPIs de Monitoring da tecnologia escolhida. Passar o mouse sobre qualquer um move o cursor vertical dos nove

juntos, no mesmo instante de tempo, e mostra um tooltip detalhado apenas no gráfico sob o mouse. Abas separam 4G e 5G, e uma barra no topo permite olhar um cluster ou um site específico.

Escopo detalhado

Composição da grade (§0.7)

Aba	Painéis
4G (9, sem par)	Acessibilidade · Availability · Drop · DL PRB · UL PRB · Interferência · Throughput DL · Throughput UL · UE médio
5G (5 sozinhos)	Acessibilidade · Drop · User Médio · Availability · UL Interference
5G (4 pares)	PRB Utility · Throughput · Traffic Volume SA · Traffic Volume NSA

Em cada par, **DL é linha e UL é barra**, com **eixo Y duplo** (esquerdo = DL, direito = UL). DL e UL compartilham unidade mas não magnitude — em escala única a barra de UL fica rente ao zero. O Chart.js 4.4.0 resolve com `type` por dataset.

api/api.py

- `get_kpi_overview(event_id, scope, scope_id, technology_family, minutes)` — **um endpoint só**, devolvendo as 13 métricas da família sobre **uma grade de tempo comum (§0.6)**:

```
{"ok": True, "labels": [...],  
  "metrics": {"accessibility": [...], "utilization_dl": [...], ...}, # null onde falta ponto  
  "units": {...}, "thresholds": {...}}
```

Uma ida ao backend em vez de nove, e o sync por índice passa a estar **correto por construção**.

`scope` aceita `site` (id fundido) ou `cluster`, reaproveitando a expansão da Fase 4.

frontend/index.html + frontend/css/main.css

- Botão `#open-kpi-overview` ("Ver KPIs") no `#chart-controls`, entre `#cell-selector` e `#time-tabs`, como no esboço.
- Modal full-screen no molde de `#chart-popup-modal` (:541): abas 4G/5G, barra de filtros (cluster + site + janela de tempo) e grid 3×3 responsivo.

frontend/js/kpi_overview.js (módulo novo)

- Definição declarativa dos 9 painéis por família (id, rótulo, unidade, par DL/UL quando houver).
- **Plugin de crosshair**: desenha a linha vertical no índice ativo em **todos** os gráficos.
- **Tooltip externo em HTML**, reaproveitando o padrão de [vip.js:394-398](#) — renderizado **só no gráfico sob o mouse**, com os valores daquele painel.
- `spanGaps: false` em todas as séries: com a grade comum, buraco de uma métrica **tem** de aparecer como buraco, não como reta ligando dois pontos distantes.
- Métrica sem nenhum ponto na janela renderiza o painel com “sem dados no período”, não um gráfico vazio sem explicação.

frontend/js/bridge.js

- Mock de `get_kpi_overview` com grade comum e buracos deliberados em uma das métricas, para o teste de sincronização rodar sem VPN.

Testes

- `test_overview_devolve_todas_as_metricas_na_mesma_grade` — o teste central do §0.6: todas as listas de `metrics` têm o mesmo comprimento de `labels`. Deve **falhar** se alguém voltar a montar cada métrica com os seus próprios timestamps.
- `test_metrica_sem_ponto_vem_como_null_e_nao_desloca_a_grade` — a métrica com 9 pontos e a com 69 saem alinhadas no mesmo eixo.
- `test_overview_por_cluster_expande_os_membros` — escopo cluster soma os sites certos.
- `test_grade_4g_tem_nove_paineis_sem_rtt` — trava a composição: `ran_rtt` / `terrestrial_rtt` fora.
- `test_grade_5g_tem_nove_paineis_e_cobre_treze_metricas` — trava os 4 pares.
- Playwright: passar o mouse num painel e afirmar que **os nove** têm crosshair no mesmo índice e que **só um** tem tooltip visível.

Como validar

```
.venv\Scripts\python.exe -m pytest tests/test_api.py tests/test_frontend_collection_ui.py -v
.venv\Scripts\python.exe -m pytest tests/ -q --basetemp=.pytest-work/tmp
```

Visualmente:

1. “Ver KPIs” abre a tela com **9 painéis** na aba 4G.
2. Passar o mouse sobre o painel de Throughput DL: os **nove** mostram a linha vertical no mesmo horário; só o painel sob o mouse mostra o tooltip detalhado.
3. Conferir o horário do crosshair contra o eixo X de dois painéis com densidade diferente (Availability e Drop) — **têm de ser o mesmo instante**.
4. Trocar para a aba 5G: 9 painéis, com PRB/Throughput/Volume SA/Volume NSA mostrando linha (DL) e barra (UL) e dois eixos Y.
5. Filtrar por cluster “Sul” → os nove painéis recarregam com o agregado do grupo.
6. Escolher uma janela em que uma métrica não tem dado → aquele painel diz “sem dados no período” e os outros oito seguem alinhados.

Sugestão de commit

feat: add KPI overview screen with nine synchronized charts

Riscos e pontos em aberto

- **Fusão por nome depende da qualidade do `nename` da EP.** A regra de 50 m protege contra homônimos distantes, mas dois sites realmente distintos com o mesmo nome **na mesma coordenada** seriam fundidos. Não há caso assim nos dados medidos; o log de fusão existe para que apareça.
- **`_merged_sites` é ponto único de falha por construção.** Se um consumidor novo ler `config["sites"]` cru, devolve id que nenhum marcador tem. Vale um teste de fumaça que percorra os consumidores conhecidos.
- **A Fase 3 não foi validada em campo com >3 tasks.** O desenho de 1 POST por task é o formato observado, mas o número de tasks simultâneas de um evento grande (~21) nunca foi exercitado contra o OSS. A guarda de duração de ciclo existe para que o limite apareça como WARNING e não como atraso silencioso.
- **Unidade de `throughput_u1` 5G segue pendente** — calcula, mas `N.ThpTime.UE.UL.RmvSmallPkt` vem sem unidade no `counterRes`. Uma comparação contra a tela do iManager fecha o `production_ready`. Não bloqueia nenhuma fase.
- **Corpo da task 2242 (NR DU Cell de OUTRAS) segue sem captura** — pendência herdada do plano de 14/08, sem impacto nas fases acima.
- **`MEMORY.md` está desatualizado** no baseline de testes (§0.10) e precisa registrar as decisões travadas no §1 ao fim de cada fase.